

University of Washington - Bothell

MyHouseHold



Kenny, Karim, Peter, Surya

CSS360

Spring 25

Software Design Document

Contents

Section 1 - Project Description	2
1.1 Project	2
1.2 Description	2
1.3 Revision History of the Document	3
Section 2 - Overview	4
2.1 Purpose.....	4
2.2 Scope.....	4
3. Requirements	4
3.1 Functional Requirements	4
3.2 Non-Functional Requirements	5
Section 4 - System Architecture	6
Section 5 - User Interface Design	8
Section 6 - Other Interfaces (optional).....	10
Section 7 - Extra Design Features / Outstanding Issues (optional).....	Error! Bookmark not defined.
Section 8 – Detailed Design.....	11
Section 9 – References	13
Section 10 – Glossary	13

Section 1 - Project Description

1.1 Project

MyHouseHold

1.2 Description

MyHousehold is a budgeting and task-management app designed to help members of a household collaboratively manage finances, purchases (e.g., groceries), and responsibilities. It offers a centralized platform for organizing shared expenses, setting budgets, and tracking tasks to make everyday life more efficient and transparent.

Project Title

MyHousehold – Collaborative Household Budgeting App

Software Design Document

What is this document about?

This document outlines the requirements and core functionality of MyHousehold, a software tool designed to help households manage shared budgets purchases, and responsibilities, it includes the product to help households manage shared budgets, purchases and responsibilities. It includes the product description, intended users, functional and non-functional requirements, and development context

What is the product?

MyHousehold is a multiplatform budgeting and takes management application tailored for households. IT allows multiple users within a home to collaboratively manage financial responsibilities, organize recording purchases like groceries, assign takes and stay within budget through shared visibility and tracking tools

Who are the product's customers/users?

The target users are:

- Families or roommates sharing a household
- Couples managing joint finances
- Any group of people living together who need to coordinate purchases, budgets, and shared responsibilities

Who created it?

This project is being developed by a student team:

- Surya
- Peter
- Karim
- Kenny

The project is assumed to be developed as part of a university course or academic software engineering assignment, without external organizational funding.

1.3 Revision History of the Document

Date	Comment	Author
5/08/25	Initial version.	Everyone
5/15/25	Requirements + Component/Module Design	Everyone
5/18/25	High-Level Architecture + Glossary	Kenny
5/18/25	Detailed Design + Finishing details	Everyone

Software Design Document

Section 2 - Overview

2.1 Purpose

Provide a budgeting and task management app that can help household members of all kinds collaborate on managing their finances and responsibilities.

2.2 Scope

The app allows users to create, set, and track budgets, manage tasks, set alerts/notifications, and optionally collaborate with other household members, with all related data being stored securely. The app will span across multiple platforms, including Windows, macOS, iOS, and Android, ensuring a seamless user experience no matter what device one might be on.

3. Requirements

Add as many as requirements your team received.

3.1 Functional Requirements

#	Description	Hrs. Est.
1	The system shall provide a login page.	8-12
2	The system should allow users to color-code different types of purchases.	8-12
3	The system shall allow users to edit and create their own budgets, tasks, and purchases.	16-24
4	The system shall allow budgets, tasks, and purchases to be organized by different categories (e.g., due date, occurrence, household member).	12-16
5	The system must store all information in a database.	8-12
6	The system should notify users of upcoming weekly/monthly purchases and tasks.	12-16
7	The system should notify users of late tasks and over-budget purchases.	12-16
8	The system shall allow for multiple users to collaborate on a household.	16-20
9	The system should allow users to keep track of their calendar.	8-12
10	The system could reward users for completing tasks and staying under budget.	12-16
11	The system could include a tutorial that guides users through the program's features.	10-14
12	The system won't incorporate AI to help users create an effective budget.	0
13	The system must log changes to budgets or purchases for accountability.	12-16
14	The system could allow users to split purchases between multiple users in a household.	8-12
15	The system shall allow users to archive completed tasks or past budgets.	8-12
	TOTAL:	159-232

Software Design Document

3.2 Non-Functional Requirements

#	Description	Hrs. Est.
1	The application must support Windows 10+, macOS 10.15+, iPhone, and Android.	24-40
2	The system must have security functions to protect user data in both the app and database.	16-24
3	The UI must allow users to perform basic operations within 30 minutes of first use.	10-16
4	The application should load in under 5 seconds on PC and mobile.	8-12
5	The system must be able to handle at least 500 users at launch.	12-20
6	The system could support localization and language switching.	12-16
7	The system should include a feedback or support feature.	8-12
8	The system should have a help section with tips and FAQs.	8-12
9	The system should minimize battery/power usage.	8-12
10	The system should have high uptime (availability).	12-16
	TOTAL:	118-180

Software Design Document

[Section 4 is the implementation of part 3 of G3 instructions.]

Section 4 - System Architecture

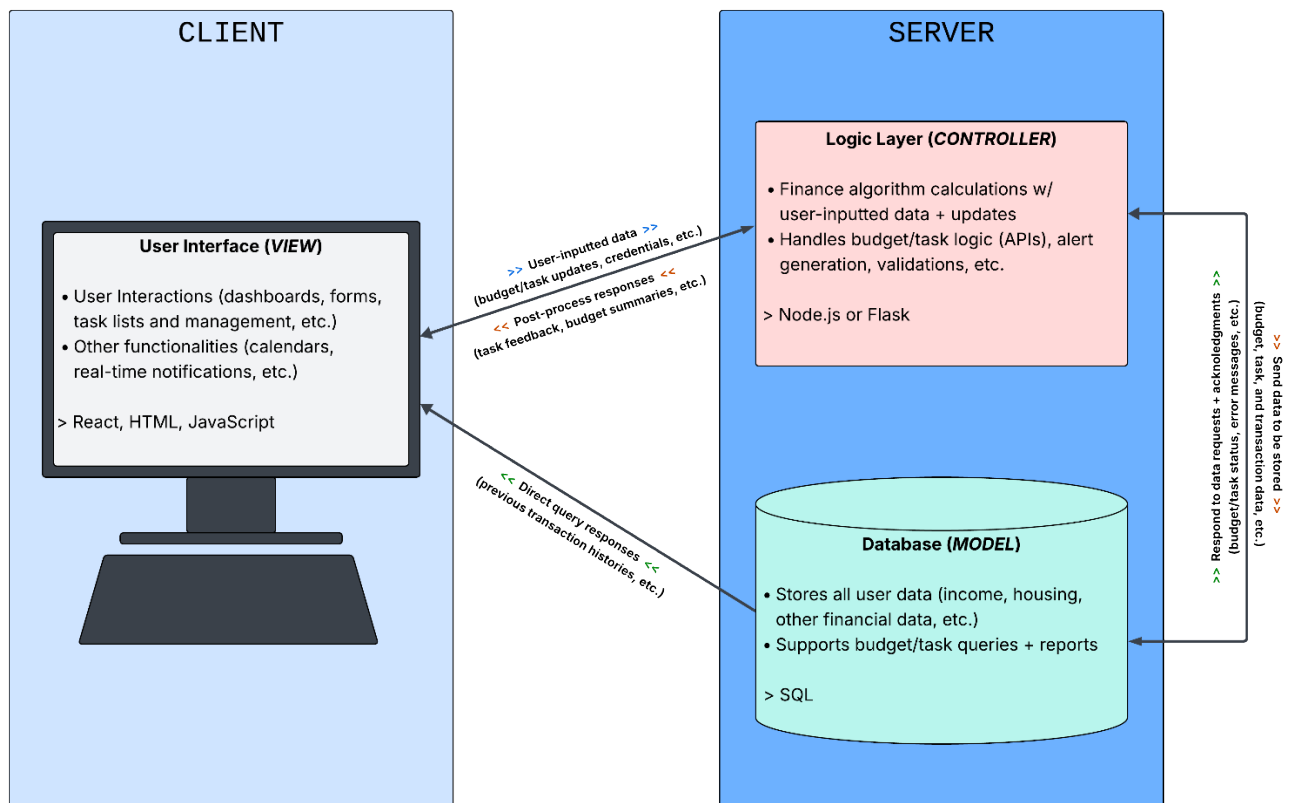
Architectural Pattern: Client–Server (MVC-based)

The MyHouseHold system adopts a client-server architecture using the MVC pattern. This structure was chosen because for its learning separation of concerns, which simplifies development.

Why This Pattern?

- Separation of concerns: Isolates UI (View), business logic (Controller), and data (Model).
- Supports concurrent access: Enables multiple household members to collaborate in real-time.
- Integrates easily with third-party services: E.g., notifications and authentication.
- Alternatives considered: Monolithic and microservice architectures were evaluated, but dismissed due to the project's limited scope.

High-Level Architecture:



Software Design Document

Component Overview

Client (User Interface)

This module enables user interaction through web and mobile interfaces. It allows household members to manage tasks, view budgets, and receive real-time updates. It addresses the functional requirements for login, task/budget editing, data visualization, and notifications.

It hides the complexities of the data structure and communication logic from the user and acts as the system's *View* in MVC.

This module represents the **presentation layer** and formats JSON responses into human-readable components.

- **Purpose:** User interaction – dashboards, forms, task lists.
- **Tech Stack:** React, HTML/CSS, JavaScript.
- **Data Flow:** Makes API requests; renders JSON responses.

Server (Controller & Logic Layer)

This module controls the application's logic and handles client-server interactions. It routes user actions (from the UI) to the correct services and enforces business rules like task deadlines, budget limits, and access control.

It addresses the requirements around secure login, role-based access, expense validation, task scheduling, and alert generation.

This module hides the control logic and rule enforcement algorithms. It serves as the Controller in the MVC structure.

- **Purpose:** Handles routing, authentication, budget/task logic.
- **Tech Stack:** Node.js/Express or Flask (Python).
- **Responsibilities:**
 - API endpoint handling
 - Business rule enforcement
 - Notifications and alerts

Database (Model Layer)

This module provides persistent, structured storage for all entities including users, households, budgets, tasks, and logs. It supports all data-centric functional requirements like task tracking, budget categorization, historical spending retrieval, and account linking.

It hides raw SQL operations and low-level database interactions, exposing only abstracted methods for storing and retrieving structured data.

The module represents the data model layer in MVC and is responsible for maintaining consistency and enforcing referential integrity.

- **Purpose:** Persistent data store for all user and household records.
- **Tech Stack:** PostgreSQL or MySQL.
- **Rationale:** Relational structure supports budget/task queries and reports.

Software Design Document

Client/User interface module -

- **Authentication module**
 - Login(String password, String usernam)
 - Register(String[] user_details)
- **Dashboard module**
 - DisplaySummary(String[] data)
 - DisplayToPay(String[] data)
- **Calendar module**
 - > displayBillDueDate(String[] dueDates)
 - > displayPaymentDate(String[] payDates)

Finance algorithm module -

- **Budget Manager module**
 - setBudgetLimit(userId, category, limit)
 - getBudgetLimits(userId)
- **Transaction Processor module**
 - addTransaction(userId, amount, category, date)
 - validateTransaction(data)
- **Report Generator module**
 - getBudgetSummary(userId, month)
 - getSpendingTrends(userId)

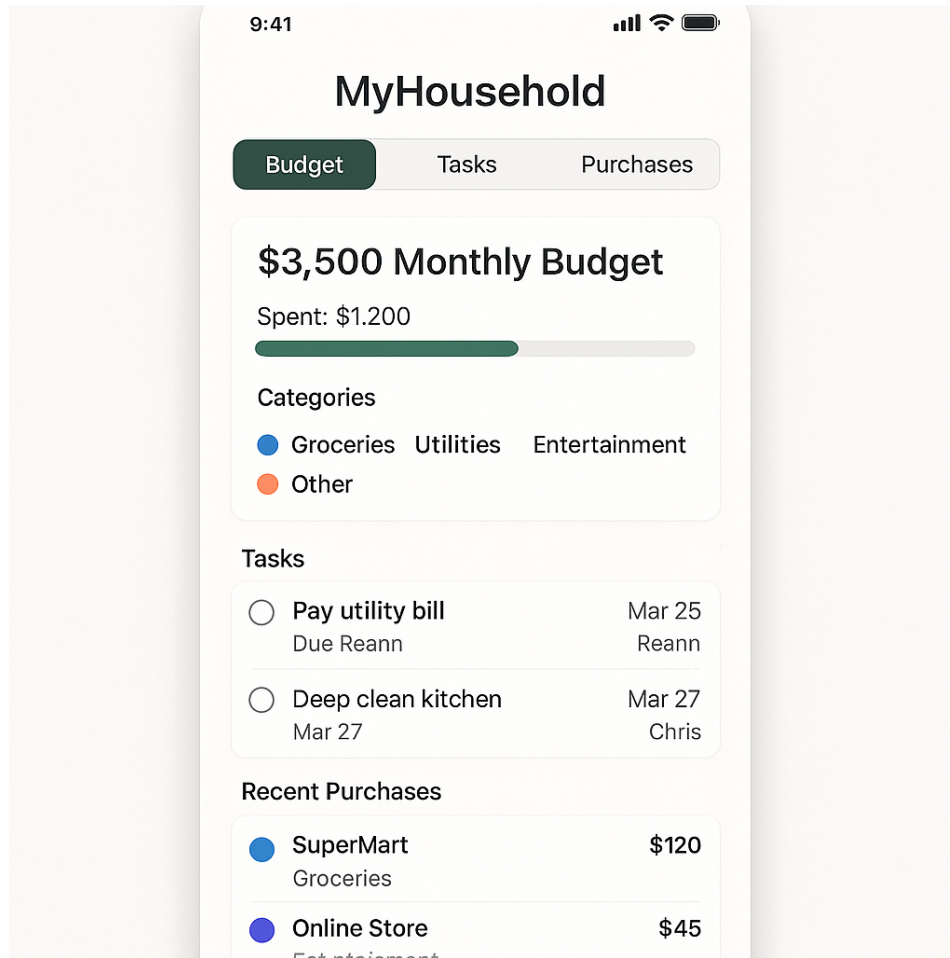
Database module-

- **Transaction Store module**
 - saveTransaction(transactionObj)
 - getTransactions(userId, month)
- **User Store module**
 - getUserData(userId)
 - saveUserData(userObj)
- **Audit Logger module**
 - logChange(entityType, entityId, changeData)
 - getLogs(filter)

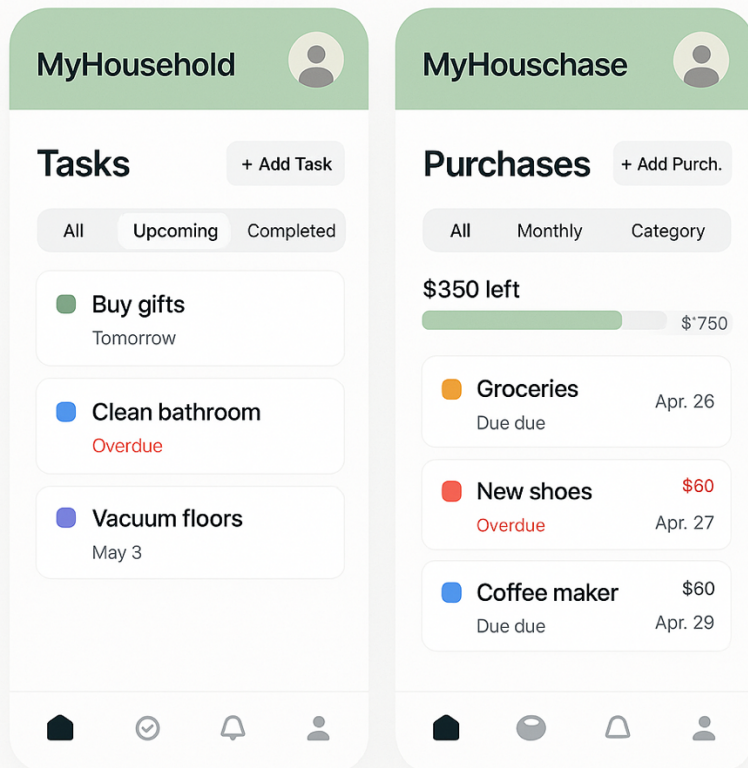
Software Design Document

Section 5 - User Interface Design

Include the main UI for the system.



Software Design Document



Software Design Document

Section 6 - Other Interfaces (optional)

Database: The application interacts with a cloud SQL database (no specific one in mind) to store user data, budgeting entries for users, and tasks.

Authentication API: We will use a third-party authentication provider like as Firebase or Auth0 and it will be used to manage BOTH user sign-in and secure access control.

Calendar: We will use use a calendar library or a third-party calendar API (maybe Google Calendar API) to allow users to track their deadlines and scheduled billings.

Notification System: We will use scheduled notifications and alerts system that is handled through a web push service or an in-app alert system that will be using JavaScript-based libraries (I could use one that we produced in our 342 class)

Currency Formatting: We can use an open-source library like Money.js (already discussed in of our sidebar assignments) to make sure there is handling of currency values and exchanges across all of our money services

Section 8 – Detailed Design

Client (User Interface)

This module enables user interaction through web and mobile interfaces. It allows household members to manage tasks, view budgets, and receive real-time updates. It addresses the functional requirements for login, task/budget editing, data visualization, and notifications.

It hides the complexities of the data structure and communication logic from the user and acts as the system's *View* in MVC.

This module represents the **presentation layer** and formats JSON responses into human-readable components.

- Purpose: User interaction – dashboards, forms, task lists.
- Tech Stack: React, HTML/CSS, JavaScript.
- Data Flow: Makes API requests; renders JSON responses.

Provided Interface

- **Authentication module**
 - Login(String password, String username)
 - Register(String[] user_details)
- **Dashboard module**
 - DisplaySummary(String[] data)
 - DisplayToPay(String[] data)
- **Calendar module**

Software Design Document

- > displayBillDueDate(String[] dueDates)
- > displayPaymentDate(String[] payDates)

Required Interface

- getUserData(userId)
- saveUserData(userObj)
- saveTransaction(transactionObj)
- getTransactions(userId, month)
- addTransaction(userId, amount, category, date)
- validateTransaction(data)

Server (Controller & Logic Layer)

This module controls the application's logic and handles client-server interactions. It routes user actions (from the UI) to the correct services and enforces business rules like task deadlines, budget limits, and access control.

It addresses the requirements around secure login, role-based access, expense validation, task scheduling, and alert generation.

This module hides the control logic and rule enforcement algorithms. It serves as the Controller in the MVC structure.

- Purpose: Handles routing, authentication, budget/task logic.
- Tech Stack: Node.js/Express or Flask (Python).
- Responsibilities:
 - API endpoint handling
 - Business rule enforcement
 - Notifications and alerts

Provided Interface

- **Transaction Store module**
 - saveTransaction(transactionObj)
 - getTransactions(userId, month)
- **User Store module**
 - getUserData(userId)
 - saveUserData(userObj)
- **Audit Logger module**
 - logChange(entityType, entityId, changeData)
 - getLogs(filter)

Required Interface

- setBudgetLimit(userId, category, limit)
- getBudgetLimits(userId)
- getBudgetSummary(userId, month)
- getSpendingTrends(userId)

Software Design Document

Database (Model Layer)

This module provides persistent, structured storage for all entities including users, households, budgets, tasks, and logs. It supports all data-centric functional requirements like task tracking, budget categorization, historical spending retrieval, and account linking.

It hides raw SQL operations and low-level database interactions, exposing only abstracted methods for storing and retrieving structured data.

The module represents the data model layer in MVC and is responsible for maintaining consistency and enforcing referential integrity.

- Purpose: Persistent data store for all user and household records.
- Tech Stack: PostgreSQL or MySQL.
- Rationale: Relational structure supports budget/task queries and reports.

Provided Interface

- **Budget Manager module**
 - setBudgetLimit(userId, category, limit)
 - getBudgetLimits(userId)
- **Transaction Processor module**
 - addTransaction(userId, amount, category, date)
 - validateTransaction(data)
- **Report Generator module**
 - getBudgetSummary(userId, month)
 - getSpendingTrends(userId)

Required Interface

None, this is the lowest layer in the system meaning that other modules call it, but it in itself does not call any other internal modules.

Section 9 – References

DALL-E AI for UI designs.

Software Design Document

Section 10 – Glossary

Acronyms:

1. **MVC:** Model-View-Controller
 - A software architectural pattern that separates an application into three interconnected components. The Model represents the data, the View is the user interface, and the Controller handles the logic and user input.
2. **UI:** User Interface
 - The space where interactions between humans and machines occur. In the context of the MyHouseHold project, the UI is the web and mobile interface through which users interact with the app.
3. **API:** Application Programming Interface
 - A set of tools and protocols that allow different software applications to communicate with each other. In MyHouseHold, APIs are used for communication between the client and server.
4. **SQL:** Structured Query Language
 - A standard programming language used for managing relational databases and performing CRUD operations.
5. **CRUD:** Create, Read, Update, Delete
 - The four basic operations for managing data in a database. CRUD operations are fundamental to the database module in the MyHouseHold system.
6. **JWT:** JSON Web Token
 - A compact, URL-safe means of representing claims to be transferred between two parties. It is commonly used for user authentication in web applications.
7. **DB:** Database
 - A structured collection of data. In the MyHouseHold system, it stores information about users, tasks, budgets, and transactions.
8. **Node.js:** Node.js
 - A JavaScript runtime built on Chrome's V8 engine, commonly used for building scalable server-side applications, such as the server component in the MyHouseHold system.
9. **REST:** Representational State Transfer
 - An architectural style for designing networked applications, used in the server's APIs for communication between the client and the database.

Key Terms:

1. **Authentication**
 - The process of verifying the identity of a user, typically through username and password or other methods like JWT tokens. Used in MyHouseHold for securing user login and access.
2. **Budgeting**
 - The practice of creating a plan to spend money within the app. The system allows users to set limits for different categories (e.g., groceries, rent) and tracks their spending accordingly.
3. **Transaction**
 - Any financial exchange or purchase within the system, tracked and categorized by users. Users can add transactions and view their spending against their budget.
4. **Notification**
 - Alerts or reminders sent to users about overdue tasks, upcoming payments, budget limits, or other relevant actions within the app.
5. **Task Management**

Software Design Document

- The feature allowing users to create, assign, and track tasks, such as bill payments, household chores, or shopping lists.
- 6. **Calendar Integration**
 - The system's ability to integrate with the user's calendar to show due dates for tasks, payments, and other time-sensitive activities.
- 7. **User Roles**
 - The system supports different user roles (e.g., Admin, Regular User) to control access and permissions. Roles allow for multi-user collaboration within a household.
- 8. **Localization**
 - The feature that enables the app to be available in different languages and formats based on the user's region, making the app more accessible.
- 9. **Security**
 - Measures to protect user data, including encryption, secure authentication, and data protection practices that safeguard the system and its users.
- 10. **Audit Log**
 - A record of all changes made within the system, such as updates to budgets, tasks, and transactions. This log is essential for accountability and traceability.

Modules/Components:

1. **Authentication Module**
 - A component of the server responsible for managing user login, registration, and session management. It handles security and ensures only authorized users can access the system.
2. **Dashboard Module**
 - Part of the UI layer, it displays a summary of a user's budget, tasks, and transactions, providing an overview of household financial status.
3. **Calendar Module**
 - This module allows users to manage dates for payments, due dates, and other tasks. It integrates with the user's calendar to show upcoming events.
4. **Finance Algorithm Module**
 - A backend module responsible for processing budgeting logic, including setting limits, tracking spending, and generating reports.
5. **Transaction Store**
 - A database component that stores all user transactions, including date, category, amount, and other transaction details.
6. **User Store**
 - A database component that stores all user information, including login credentials, preferences, and personal data.
7. **Audit Logger**
 - A component that tracks and logs every change made to the system, such as budget adjustments or task completions, ensuring traceability.
8. **Notification System**
 - The part of the app that sends users alerts about tasks, purchases, or important events, helping them stay on track with their financial responsibilities.
9. **Report Generator**
 - A module in the backend that generates detailed reports on budget status, spending trends, and transaction history, giving users insight into their financial habits.

Technologies/Tools:

1. **React**
 - A JavaScript library used to build user interfaces for the web. It is the main framework used for building the UI layer in MyHouseHold.

Software Design Document

2. **Node.js/Express**

- A runtime and framework used for building the server-side logic of MyHouseHold. It handles API requests, user interactions, and business logic.

3. **PostgreSQL/MySQL**

- Relational database management systems used to store data persistently in MyHouseHold, including user profiles, budgets, tasks, and transactions.

4. **JWT (JSON Web Tokens)**

- A method of securely transmitting information between parties in the app. Used for user authentication in MyHouseHold.

5. **Flask**

- A lightweight Python web framework that can be used as an alternative to Node.js/Express for building the server-side components of MyHouseHold.